Universidade Estadual da Paraíba

Centro de Ciências e Tecnologia

Departamento de Computação

Disciplina: Técnicas de Análise de Algoritmos

Professor: Fábio Luiz Leite Júnior

Emanuel Diogo Marinho de Souza

Filipe Antonny Tavares de Santana

Rafael Trajano do Nascimento

Projeto de Técnicas de Análise de Algoritmos

Problema: Alocação de Salas de Aula

Campina Grande – PB

18 de março de 2026

Sumário

1. Introdução……3
2. Metodologia e implementação…….4
3. Algoritmos implementados……...5
   3.1 Backtracking……5
      3.1.1 Análise de corretude……6
      3.1.2 Análise de desempenho…….….7
         3.1.2.1 Espacial……...7
         3.1.2.2 Temporal…….….7
   3.2 Branch and Bound…….8
      3.2.1 Análise de corretude……...8
      3.2.2 Análise de desempenho…….9
         3.2.2.1 Espacial……..9
         3.2.2.2 Temporal……..…..….9
4. Experimentos e resultados…….10
5. Análise e discussão…….14
6. Conclusão…….16
7. Referências……...17

1. Introdução

O problema de alocação de salas de aula consiste em determinar o número mínimo de salas necessárias para acomodar um conjunto de aulas, considerando seus horários de início e término. Duas aulas que possuem horários sobrepostos não podem ocorrer na mesma sala. Esse tipo de problema aparece em diversas áreas, como planejamento acadêmico, organização de eventos e escalonamento de tarefas.

Neste trabalho são analisadas duas técnicas de projeto de algoritmos: Backtracking e Branch and Bound (CORMEN et al., 2009). Ambas foram implementadas na linguagem Java para resolver o problema de alocação de salas de aula. O objetivo é comparar as estratégias utilizadas por cada técnica e avaliar suas características em termos de funcionamento e eficiência.

2. Metodologia e Implementação

O sistema foi desenvolvido utilizando a linguagem de programação Java. O programa recebe como entrada o número de aulas e os horários de início e término de cada uma delas. A partir dessas informações, os algoritmos implementados determinam o número mínimo de salas necessárias e realizam a alocação das aulas em salas disponíveis.

A entrada do programa consiste em um número inteiro n, representando a quantidade de aulas, seguido de n linhas contendo dois valores inteiros: horário de início e horário de término da aula.

A saída apresenta o número mínimo de salas necessárias e a atribuição de cada aula a uma sala específica.

3. Algoritmos Implementados

3.1 Backtracking

O algoritmo de Backtracking explora sistematicamente todas as possíveis combinações de alocação das aulas em salas (KLEINBERG; TARDOS, 2006). Para cada aula, o algoritmo tenta colocá-la em uma das salas já utilizadas. Caso haja conflito de horário com outra aula presente na mesma sala, o algoritmo tenta outra sala. Caso nenhuma sala seja possível, o algoritmo cria uma nova sala e continua a busca.

```
private void backtrack(Aula[] aulas, int[] salas, int
aulaAtual, int salasUsadas) {

   if (aulaAtual == aulas.length) {

       if (salasUsadas < melhorSalas) {
           melhorSalas = salasUsadas;
           melhorSolucao = salas.clone();
       }

       return;
   }

   for (int s = 1; s <= salasUsadas; s++) {

       if (podeColocar(aulas, salas, aulaAtual, s)) {

           salas[aulaAtual] = s;

           backtrack(aulas, salas, aulaAtual + 1,
salasUsadas);
       }
   }
}
```

Nesse trecho do código é possível observar o processo recursivo do Backtracking. O algoritmo tenta alocar a aula atual em cada uma das salas disponíveis. Caso a alocação seja válida, ele continua a busca para a próxima aula.

3.1.1 Análise de Corretude

O algoritmo de Backtracking garante a corretude da solução ao explorar sistematicamente todas as combinações possíveis de alocação das aulas em salas (CORMEN et al., 2009). A cada passo da recursão, o algoritmo verifica se a alocação da aula atual em determinada sala é válida, ou seja, se não existe conflito de horário com outras aulas já atribuídas à mesma sala.

Caso a alocação seja válida, o algoritmo prossegue recursivamente para a próxima aula. Caso contrário, outra sala é testada. Esse processo continua até que todas as aulas sejam alocadas.

Como o algoritmo percorre todo o espaço de busca de soluções possíveis, nenhuma combinação válida é ignorada. Dessa forma, é garantido que a solução encontrada corresponde ao número mínimo de salas necessárias, caracterizando uma solução ótima para o problema.

No que se refere à condição de parada da recursão, ela ocorre quando todas as aulas já tenham sido processadas e atribuída uma sala a cada aula. Isso é verificado pela condição `if (aulaAtual == aulas.length)` . Nesse momento, significa que o algoritmo conseguiu construir uma solução completa, ou seja, cada posição do vetor *salas* contém a sala atribuída para a aula correspondente. Ao atingir essa situação, o algoritmo compara o número de salas utilizadas na solução atual (*salasUsadas*) com a melhor solução encontrada até então (*melhorSalas*). Caso a solução atual utilize menos salas, ela passa a ser armazenada como a melhor solução, sendo copiada para *melhorSolucao*. Após essa verificação, a função retorna, encerrando aquela ramificação da busca recursiva. Dessa forma, a condição de parada garante que o algoritmo finalize corretamente quando todas as aulas tiverem sido consideradas, e a melhor opção tiver sido armazenada.

A manutenção da invariante no algoritmo ocorre durante a atribuição de salas para as aulas ao longo da execução recursiva. A invariante estabelece que, antes de cada chamada recursiva, todas as aulas já processadas (da posição *0* até *aulaAtual-1*) estão alocadas em salas sem conflitos entre aulas que **não podem ocorrer no mesmo horário na mesma sala** em questão. Para garantir que essa propriedade continue verdadeira, o algoritmo utiliza a função *podeColocar*, que verifica se a aula atual pode ser colocada em determinada sala sem gerar conflito com as aulas já alocadas. Somente quando essa verificação retorna verdadeiro a sala é atribuída à aula atual e o algoritmo avança para a próxima chamada recursiva. Dessa forma, cada nova atribuição preserva a validade da solução parcial, garantindo que o invariante seja mantido ao longo de toda a execução do algoritmo.

Com isso conclui-se que o algoritmo é correto pois mantém a invariante da solução válida durante toda a sua execução, não permite conflito entre aulas, explora todas as combinações possíveis, possui um critério de parada aceitável evitando que o algoritmo execute em looping infinito e mantém sempre a melhor solução encontrada.

3.1.2 Análise de Desempenho

3.1.2.1 Espacial

A complexidade espacial do algoritmo está relacionada ao armazenamento das aulas e da estrutura utilizada para registrar a alocação das salas. O algoritmo utiliza vetores para armazenar essas informações.

Além disso, o algoritmo utiliza chamadas recursivas cuja profundidade máxima é igual ao número de aulas, uma vez que a cada nível da recursão uma nova aula é processada.

Dessa forma, a complexidade espacial do algoritmo é $O(n)$, onde n representa o número de aulas.

3.1.2.2 Temporal

O algoritmo de Backtracking pode explorar diversas combinações possíveis de alocação das aulas em salas. Para cada aula, o algoritmo tenta alocá-la em uma das salas disponíveis ou criar uma nova sala, gerando diferentes ramificações na árvore de busca.

No pior caso, o algoritmo pode explorar um número exponencial de combinações, uma vez que cada aula pode potencialmente ser alocada em diferentes salas. Dessa forma, o tamanho da árvore de busca cresce rapidamente conforme o número de aulas aumenta.

Assim, a complexidade temporal do algoritmo pode ser considerada exponencial, aproximadamente $O(k^n)$, onde k representa o número de salas possíveis e n o número de aulas (CORMEN et al., 2009).

3.2 Branch and Bound

O algoritmo Branch and Bound utiliza uma estratégia de busca semelhante ao Backtracking, explorando o espaço de soluções possíveis através de uma árvore de busca **(KLEINBERG; TARDOS, 2006)**. Entretanto, adiciona um mecanismo de poda que permite eliminar ramos da árvore que não podem produzir soluções melhores do que a melhor solução encontrada até o momento.

```
private void branchBound(Aula[] aulas, int[] salas, int aulaAtual, int
salasUsadas) {

   if (salasUsadas >= melhorSalas) {
       return;
   }

   if (aulaAtual == aulas.length) {

       melhorSalas = salasUsadas;
       melhorSolucao = salas.clone();
       return;
   }
}
```

A principal diferença em relação ao Backtracking está na utilização de um limite (bound) que representa a melhor solução já obtida. Durante a exploração da árvore de busca, sempre que uma solução parcial utiliza um número de salas maior ou igual ao melhor resultado encontrado, o algoritmo interrompe a exploração daquele ramo, descartando-o antecipadamente. Dessa forma, evita-se o processamento de combinações que não podem levar a soluções melhores.

3.2.1 Análise de Corretude

A corretude do algoritmo pode ser demonstrada nesse caso analisando três aspectos principais: condição de parada, manutenção da solução válida e poda da busca.

Primeiramente, o algoritmo possui uma condição de poda representada pela instrução:

```
   if (salasUsadas >= melhorSalas) {
       return;
   }
```

Essa verificação impede que o algoritmo continue explorando soluções parciais que já utilizam um número de salas maior ou igual à melhor solução encontrada até o

momento. Dessa forma, essa poda não compromete a corretude do algoritmo, apenas reduz o espaço de busca desejado.

A condição de parada ocorre quando todas as aulas já foram processadas, isto é, quando *aulaAtual* == *aulas.length*. Nesse momento, significa que uma solução completa foi construída. Se essa solução utiliza menos salas do que a melhor solução conhecida, ela passa a ser armazenada como a nova melhor solução.

Além disso, o algoritmo constrói a solução de forma incremental, atribuindo aulas às sal**as** uma por uma. Durante esse processo, apenas atribuições válidas são consideradas (verificadas por funções auxiliares como *podeColocar*), garantindo que nenhuma aula conflitante seja colocada na mesma sala. Assim, a cada passo, a solução parcial mantém a propriedade de validade. Isso faz com que o algoritmo explore sistematicamente as possíveis combinações de atribuição de salas, eliminando apenas aquelas que não podem levar a uma solução melhor. Como consequência, quando o algoritmo termina, a variável *melhorSolucao* contém uma alocação válida de salas que minimiza o número de salas utilizadas.

Então conclui-se que o algoritmo é correto, pois mantém soluções parciais válidas, utiliza poda segura para reduzir o espaço de busca e garante que a melhor solução encontrada seja armazenada ao final da execução.

3.2.2 Análise de Desempenho

3.2.2.1 Espacial

Assim como no Backtracking, o algoritmo Branch and Bound utiliza estruturas simples para armazenar a alocação das aulas e as salas. Além disso, o algoritmo utiliza chamadas recursivas cuja profundidade máxima é limitada pelo número de aulas processadas. Dessa forma, sua complexidade espacial também pode ser considerada O(n), onde n representa o número de aulas.

3.2.2.2 Temporal

No pior caso, o algoritmo Branch and Bound possui complexidade temporal semelhante ao Backtracking, podendo chegar a $O(k^n)$. Entretanto, devido à estratégia de poda utilizada, muitos caminhos de busca são descartados antecipadamente, reduzindo significativamente o número de combinações exploradas na prática, tornando o algoritmo mais eficiente do que o Backtracking em muitos casos.

4. Experimentos e Resultados

Para avaliar o funcionamento dos algoritmos implementados, foi utilizado um conjunto de aulas com horários de início e término definidos. O objetivo foi verificar quantas salas seriam necessárias para acomodar todas as aulas sem que ocorressem conflitos de horário.

A primeira entrada utilizada no experimento foi a seguinte:

```
6
9  10
9  12
11 13
12 14
13 15
14 16
```

O número inteiro da primeira linha é a quantidade de aulas a ser alocadas nas salas, as linhas seguintes representam cada aula, contendo o horário de início e o horário de término.

Ao executar o algoritmo de **Backtracking**, o programa produziu o seguinte resultado:

```
2
1 1
2 2
3 1
4 2
5 1
6 2
```

O resultado indica que foram necessárias **2 salas** para acomodar todas as aulas. As linhas seguintes no resultado anterior mostram a alocação de cada aula em uma sala específica.

Ao executar o algoritmo de **Branch and Bound**, foi obtido o mesmo resultado:

```
2
1 1
2 2
3 1
4 2
```

5 1
6 2

Observa-se que ambos os algoritmos encontraram a mesma solução ótima, utilizando apenas duas salas para acomodar todas as aulas sem sobreposição de horários.

Os resultados demonstram que as duas abordagens são capazes de resolver corretamente o problema de alocação de salas de aula, garantindo que aulas com horários conflitantes não sejam atribuídas à mesma sala.

O problema é que quando o problema começa a ser escalado (aumento do número de aulas), o algoritmo que usa o Backtracking começa a ter problemas de tempo de processamento graças a seu tipo de abordagem. Foram feitos alguns testes para medir o comportamento dos algoritmos quanto a escalabilidade, para o testes foram implementados o método System.nanoTime() na classe main para calcular o tempo de execução de cada algoritmo de alocação de salas e um contador no classe BranchBoundSolver para calcular quantas podas foram feitas na execução do algoritmo Branch and Bound.

Os testes foram feitos para 10 aulas e 20 aulas e a entrada de cada teste e os seus resultados estão abaixo.

```
10
7 8
7 9
9 10
9 11
9 14
7 8
10 12
10 11
7 8
8 9
```

```
Backtracking:
4
1 1
2 2
3 1
4 2
5 3
6 3
7 1
8 4
9 4
10 1
Tempo: 0,002345 segundos
```

```
Branch and Bound:
4
1 1
2 2
3 1
4 2
5 3
6 3
7 1
8 4
9 4
10 1
Podas: 26
Tempo: 0,000670 segundos
```

Imagem 1: Teste feito com 10 aulas

Percebemos que o resultado para 10 aulas a serem alocadas é o mesmo para os dois tipos de algoritmo, os dois executam rapidamente o processo, porém já vemos que o tempo de execução do Backtracking já é um pouco maior que o Branch and Bound que consegue fazer 26 podas no seu processamento. A diferença é pequena pois o problema ainda é simples.

```
20
7 8
7 9
9 12
8 9
10 12
13 14
16 18
10 12
7 9
8 10
7 9
10 12
20 22
7 9
10 11
10 12
12 13
13 15
7 11
11 13
```

```
Backtracking:
 7
1 1
2 2
3 1
4 1
5 2
6 1
7 1
8 3
9 3
10 4
11 5
12 4
13 1
14 6
15 5
16 6
17 1
18 2
19 7
20 5
Tempo: 1120,438702 segundos
```

```
Branch and Bound:
7
1 1
2 2
3 1
4 1
5 2
6 1
7 1
8 3
9 3
10 4
11 5
12 4
13 1
14 6
15 5
16 6
17 1
18 2
19 7
20 5
Podas: 10190683
Tempo: 0,463788 segundos
```

Imagem 2: Teste feito com 20 aulas

Já na saída com 20 aulas a serem alocadas, continua com a mesma saída nos dois algoritmos. Quanto ao tempo de processamento do Backtracking teve um crescimento exponencial que foi de mais de 18 minutos enquanto o Branch and

Bound consegue o mesmo resultado com menos de meio segundo de execução e incríveis 10 milhões de podas.

Percebemos que com o aumento da escalabilidade do problema o Backtracking fica inviável de ser usado já o Branch and Bound executa em tempo hábil e apresenta uma saída satisfatória.

Falando sobre o consumo de memória, para 10 aulas o mesmo não teve mudança significativa percebida. Já para 20 salas o consumo que estava sempre entre 35% e 40% subiu para 95% no Backtracking conforme ilustrado na imagem abaixo. No caso do Branch and Bound ficou inviável visualizar se houve alteração no consumo de memória já que o mesmo foi processado em menos de meio segundo. As configurações do computador que foram feitos os testes são: processador i3 111gG4, 4GB RAM, Placa vídeo Intel 128M, SSD 238GB.

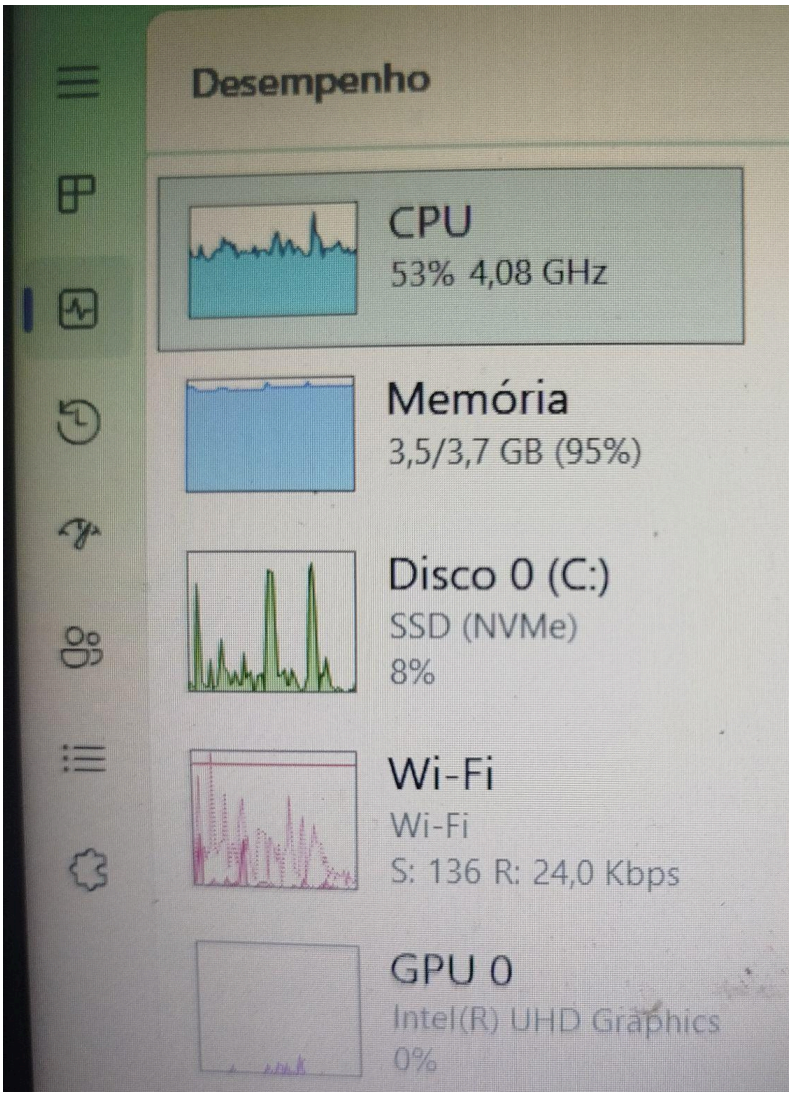

Imagem 3: Consumo de memória durante a execução do Backtracking para 20 aulas

5. Análise e Discussão

Os resultados obtidos mostram que tanto o algoritmo de Backtracking quanto o algoritmo de Branch and Bound foram capazes de resolver corretamente o problema de alocação de salas de aula. Em ambos os casos, os algoritmos encontraram a solução ótima utilizando duas salas para acomodar todas as aulas sem que ocorressem conflitos de horário.

Apesar de produzirem o mesmo resultado para o conjunto de dados utilizado, as duas técnicas possuem diferenças importantes em relação à forma como exploram o espaço de soluções. O algoritmo de Backtracking realiza uma busca completa, testando diferentes combinações possíveis de alocação de aulas em salas, conforme descrito por Kleinberg e Tardos (2006). Essa abordagem garante que todas as possibilidades sejam analisadas, porém pode resultar em maior tempo de execução quando o número de aulas aumenta.

Por outro lado, o algoritmo Branch and Bound utiliza uma estratégia de poda que evita a exploração de soluções que não podem produzir resultados melhores do que a melhor solução encontrada até o momento. Dessa forma, o algoritmo reduz o número de combinações analisadas, tornando o processo de busca potencialmente mais eficiente.

A tabela a seguir apresenta uma comparação geral entre as duas técnicas utilizadas:

Comparação entre Backtracking e Branch and Bound

Critério	Backtracking	Branch and Bound
Estratégia	Exploração completa das soluções	Exploração com poda de ramos
Eficiência	Pode apresentar maior tempo de execução em problemas maiores	Mais eficiente devido à poda
Complexidade	Alta para grandes conjuntos de dados	Reduzida em relação ao Backtracking

De modo geral, observa-se que o Branch and Bound tende a apresentar melhor desempenho em problemas maiores, pois evita a exploração de caminhos desnecessários durante a busca por soluções.

Esse comportamento também evidencia a escalabilidade dos algoritmos, uma vez que o Backtracking tende a apresentar crescimento exponencial com o aumento do número de aulas, enquanto o Branch and Bound reduz esse crescimento por meio de podas.

6. Conclusão

Neste trabalho foi analisado o problema de alocação de salas de aula, cujo objetivo é determinar o número mínimo de salas necessárias para acomodar um conjunto de aulas sem que ocorram conflitos de horário. Para resolver esse problema, foram implementadas duas técnicas clássicas de projeto de algoritmos: Backtracking e Branch and Bound, utilizando a linguagem de programação Java.

Os resultados obtidos demonstraram que ambos os algoritmos são capazes de encontrar soluções corretas para o problema proposto. Nos experimentos realizados, as duas abordagens encontraram a mesma solução ótima, utilizando duas salas para acomodar todas as aulas analisadas.

A principal diferença entre os algoritmos está na forma como exploram o espaço de soluções. O Backtracking realiza uma busca completa, analisando diferentes combinações possíveis, enquanto o Branch and Bound utiliza uma estratégia de poda que evita a exploração de soluções que não podem produzir resultados melhores, tornando-o potencialmente mais eficiente em problemas de maior escala.

De modo geral, a comparação entre as duas técnicas permitiu compreender melhor as características de cada abordagem e como elas podem ser aplicadas na resolução de problemas de otimização e escalonamento.

Dessa forma, este trabalho evidencia a importância das técnicas de projeto de algoritmos na resolução de problemas reais, destacando o impacto da escolha da estratégia na eficiência computacional.

7. Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.

KLEINBERG, J.; TARDOS, É. Algorithm Design. Boston: Pearson, 2006.